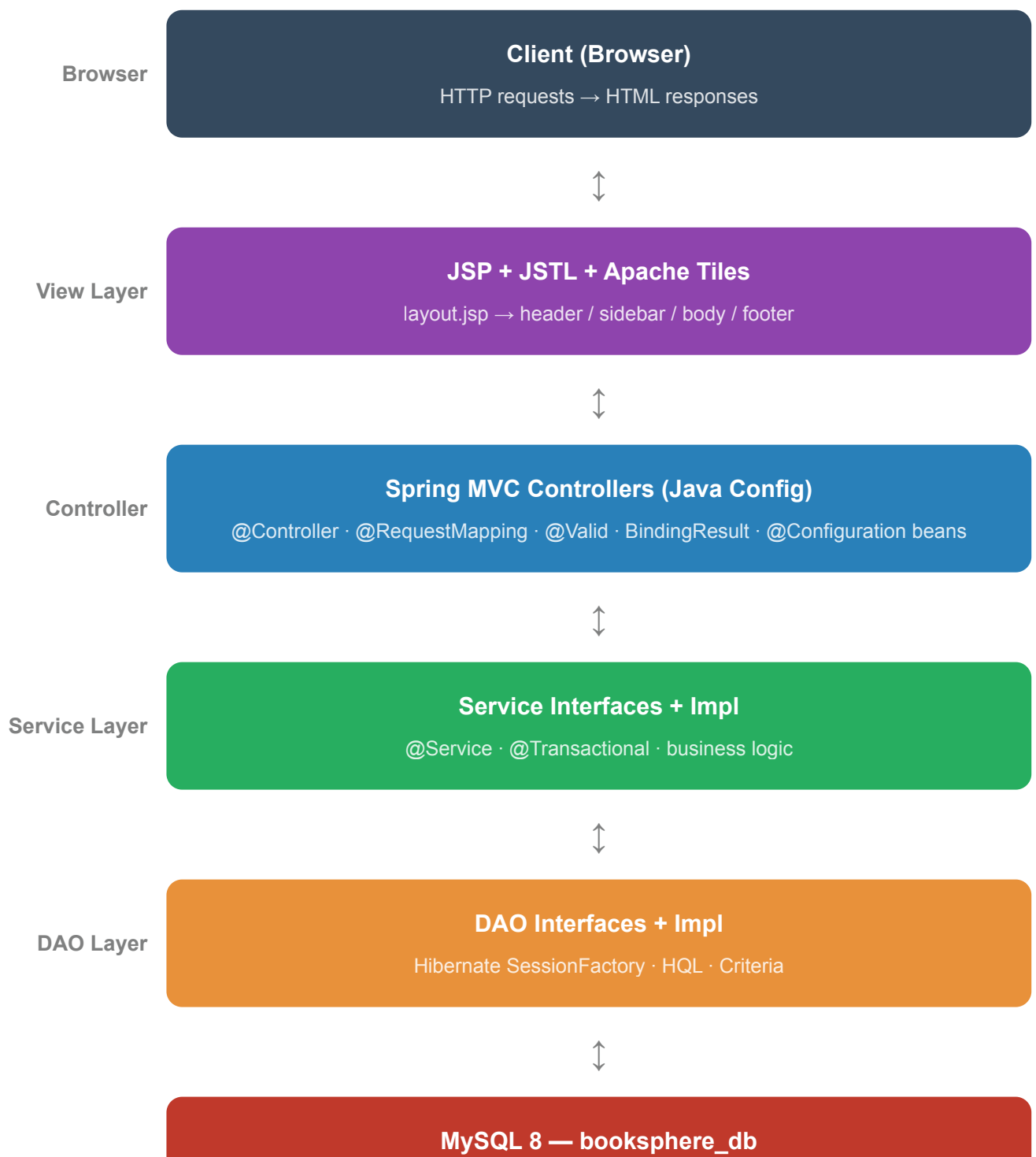


BookSphere

High-Level Design — Student Management + Library System built with
Spring MVC (Java Config), JSP/JSTL, Hibernate, Apache Tiles & Bootstrap 5

Scaffolded via [maven-archetype-webapp](#)

1. Layered Architecture



1b. Java Config (No XML Beans)

All Spring wiring is done via `@Configuration` classes in `com.booksphere.config`. The only XML file remaining is `tiles.xml` (required by Apache Tiles).

Class	Annotations / Extends	Responsibility
WebAppInitializer	extends <code>AbstractAnnotation- ConfigDispatcher- ServletInitializer</code>	Replaces <code>web.xml</code> <code>DispatcherServlet</code> declaration. Maps servlet to <code>/</code> , registers root & servlet contexts.
AppConfig	<code>@Configuration</code> , <code>@EnableTransaction- Management</code> , <code>@ComponentScan</code>	Defines <code>DataSource</code> → <code>LocalSessionFactoryBean</code> → <code>HibernateTransactionManager</code> . All Hibernate properties set here.
WebConfig	<code>@Configuration</code> , <code>@EnableWebMvc</code> , implements <code>WebMvcConfigurer</code>	Registers <code>TilesConfigurer</code> (points to <code>/WEB-INF/tiles.xml</code>), <code>TilesViewResolver</code> , and static resource handlers.

1c. Maven BOM & Jakarta Migration

Spring Framework BOM

Declared in `<dependencyManagement>` with `<scope>import</scope>`. Locks all Spring modules to **6.2.17** automatically.

Result: `spring-webmvc`, `spring-orm`, `spring-context`, `spring-tx` all omit `<version>` — the BOM supplies it.

Jakarta EE 9+ Migration

<code>javax.servlet.*</code>	→	<code>jakarta.servlet.*</code>
<code>javax.persistence.*</code>	→	<code>jakarta.persistence.*</code>
<code>javax.validation.*</code>	→	<code>jakarta.validation.*</code>
Tomcat 9	→	Tomcat 10.1+

Java 17+ is the minimum baseline for Spring 6.

2. Tech Stack

Maven + BOM

Scaffolded via maven-archetype-webapp. Uses **Spring Framework BOM** in `<dependencyManagement>` to lock all Spring modules to 6.2.17.

Spring MVC 6.2.17

Java-based config — `@Configuration`, `@EnableWebMvc`, `WebApplicationInitializer`. Jakarta EE 9+ namespace. No XML bean files.

JSP 3.0 + JSTL 3.0

Jakarta namespace. Server-side views with core tags (`c:`) and formatting tags (`fmt:`)

Apache Tiles 3

Template composition — shared header, sidebar, footer across all pages

Hibernate 6.x

ORM with Jakarta Persistence annotations, HQL, lazy/eager fetch, transaction management

Hibernate Validator 8.x

Bean Validation 3.0 (Jakarta) — `@NotBlank`, `@Email`, `@Past`, `@Size`, `@DecimalMin`

MySQL 8

Relational database with FK constraints, indexes, and sample seed data

SLF4J + Log4j2

Structured logging — console + rolling file appenders

Bootstrap 5

CDN-only responsive styling — tables, forms, nav, pagination, alerts

Tomcat 10.1+

Jakarta EE 9+ servlet container (`jakarta.*` namespace)

3. Entity Model

Student

PK id : Long

firstName : String

lastName : String

Author

PK id : Long

name : String

Category

PK id : Long

name : String

email : String

enrollmentDate : Date

department : String

gpa : Double

biography : String

description : String

Book

PK id : Long

FK author_id : Long

title : String

isbn : String

publishedYear : int

copies : int

BorrowRecord

PK id : Long

FK student_id : Long

FK book_id : Long

borrowDate : Date

returnDate : Date

status : String

book_category (Join Table)

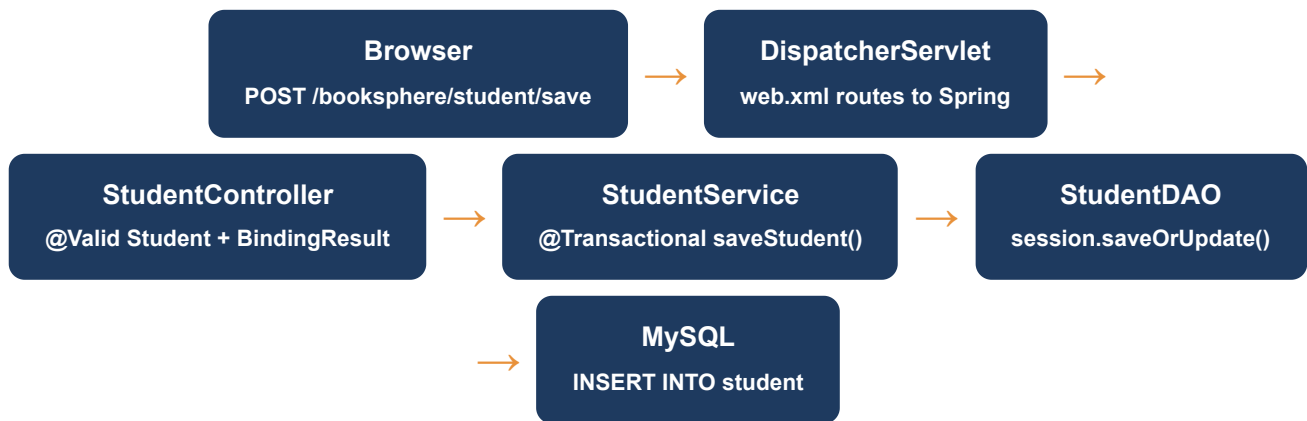
FK book_id : Long

FK category_id : Long

4. Relationships

From	To	Type	Description
Book	Author	@ManyToOne	Each book has one author; an author can have many books
Book	Category	@ManyToMany	A book belongs to multiple categories; a category contains many books. Uses book_category join table
BorrowRecord	Student	@ManyToOne	A borrow record belongs to one student; a student can have many borrow records
BorrowRecord	Book	@ManyToOne	A borrow record references one book; a book can appear in many borrow records

5. Request Flow (example: Add Student)



6. Package & File Structure

```

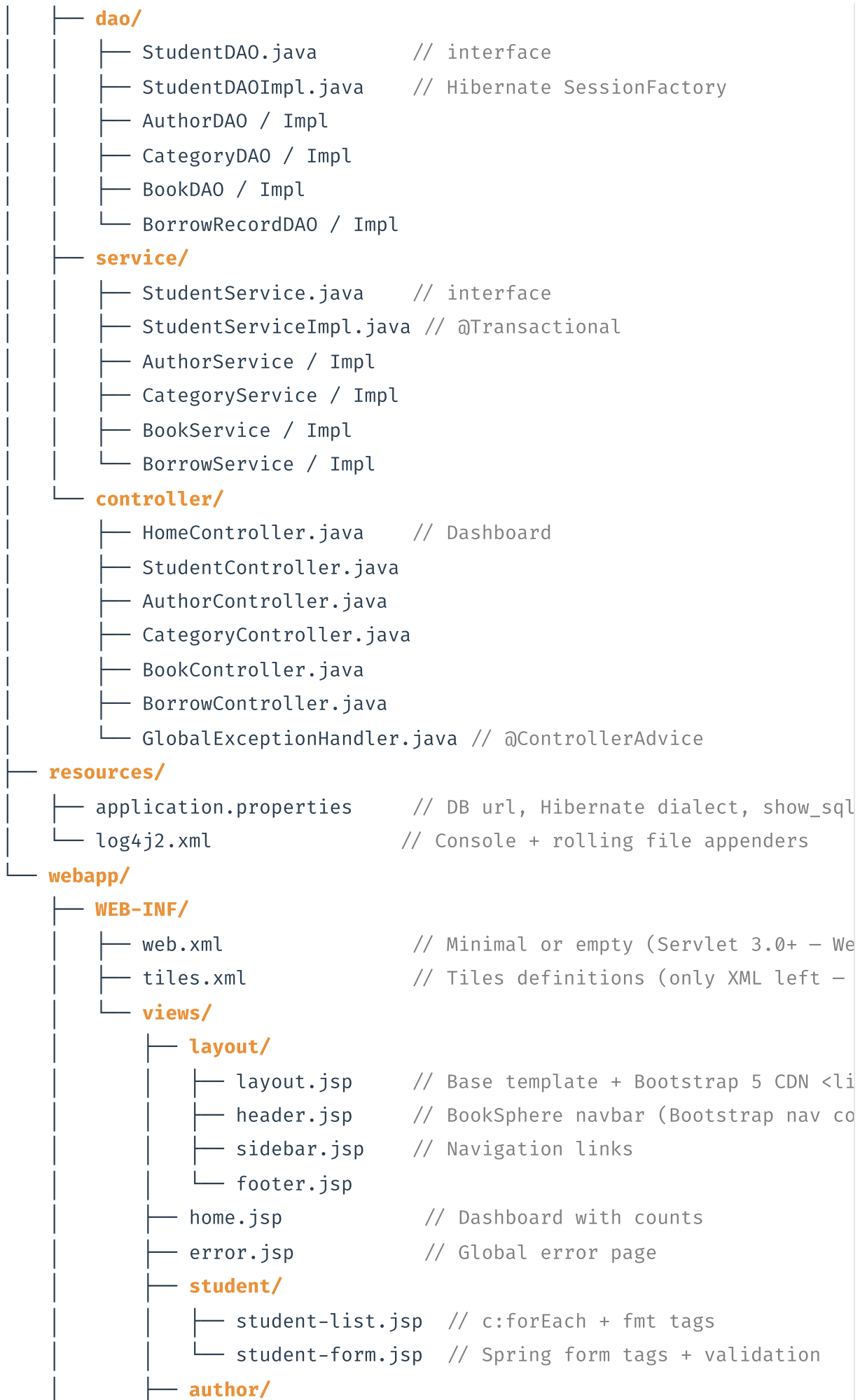
booksphere/
├── pom.xml
├── src/
│   └── main/
│       ├── java/com/booksphere/
│       │   ├── config/
│       │   │   ├── AppConfig.java
│       │   │   ├── WebConfig.java
│       │   │   └── WebAppInitializer.java
│       │   └── entity/
│       │       ├── Student.java
│       │       ├── Author.java
│       │       ├── Category.java
│       │       ├── Book.java
│       │       └── BorrowRecord.java
  
```

// Generated via: mvn archetype:generate -Da
// Java 11+, Spring MVC, Hibernate, JSTL,

// ★ Java-based configuration – NO XML
// @Configuration, @EnableTransactionM
// DataSource, SessionFactory, T
// WebMvcConfigurer – TilesConfigurer,
// AbstractAnnotationConfigDispatcherS
// Replaces DispatcherServlet in

// @Entity, JPA annotations

// @ManyToOne, @ManyToMany



```

├── author-list.jsp
├── author-form.jsp
├── category/
│   ├── category-list.jsp
│   └── category-form.jsp
├── book/
│   ├── book-list.jsp
│   └── book-form.jsp           // Author dropdown + category check
├── borrow/
│   ├── borrow-list.jsp
│   └── borrow-form.jsp
└── resources/                  // Static assets (if any)

```

7. Module Roadmap

- Module 1 — Project Skeleton & Maven Setup**
 maven-archetype-webapp scaffold, pom.xml with **Spring BOM 6.2.17**, Java 17, Jakarta migration, Java config (AppConfig + WebConfig + WebAppInitializer), Tiles layout, Log4j2, Bootstrap 5 CDN — "Hello BookSphere" page
- Module 2 — Student Entity & Hibernate Basics**
 Student entity, DAO pattern, HQL, @Transactional, MySQL schema
- Module 3 — Student List & JSTL Display**
 @Controller, Model, c:forEach, fmt:formatDate, Bootstrap table
- Module 4 — Add & Update Student (Forms + Validation)**
 Spring form tags, Bean Validation, @Valid, BindingResult, PRG pattern
- Module 5 — Delete Student & Confirmation**
 DELETE operation, JS confirmation, redirect after delete
- Module 6 — Search & Pagination**
 HQL LIKE queries, parameter binding, setFirstResult/setMaxResults
- Module 7 — Author & Category Entities**
 Reinforce the entity → DAO → service → controller → JSP pattern
- Module 8 — Book Entity with Relationships**
 @ManyToOne, @ManyToMany, @JoinTable, EAGER vs LAZY fetch

- **Module 9 — Borrowing System**
BorrowRecord associative entity, student ↔ book, return flow
- **Module 10 — Exception Handling & Logging**
@ExceptionHandler, @ControllerAdvice, error.jsp, Log4j2 in practice
- **Module 11 — Dashboard & Polish**
Flash attributes, aggregate HQL, final cleanup, README

8. Key Application URLs

URL	Method	Description
/booksphere/	GET	Dashboard — counts of students, books, active borrows
/booksphere/student/list	GET	List all students (with search & pagination)
/booksphere/student/showForm	GET	Add new student form
/booksphere/student/save	POST	Save / update student
/booksphere/student/delete	GET	Delete student by ID
/booksphere/book/list	GET	List all books with author & category badges
/booksphere/book/showForm	GET	Add/edit book — author dropdown + category checkboxes
/booksphere/borrow/list	GET	All borrow records
/booksphere/borrow/new	GET	Borrow a book form
/booksphere/borrow/return	POST	Return a borrowed book